

Punto 3.5 - Consultas de agregación: Count, Sum, Average, Min y Max

Práctica

Ejercicio: Añadir métodos de agregación al repositorio de órdenes del proyecto AceriaData. Crear métodos que usen Count, Sum, Average, Min, Max y GroupBy. Crear un caso de uso que ejecute estas agregaciones, analice el SQL generado y compare el rendimiento de agregar en el servidor versus agregar en memoria.

Contexto del proyecto: En el punto 3.4 se añadieron proyecciones a DTOs, incluyendo colecciones de navegación y DTOs anidados. En este punto se añaden consultas de agregación que calculan resúmenes directamente en el servidor. Estas agregaciones se usarán en el punto 3.6 para las agrupaciones con proyección y en el Módulo 4 para las optimizaciones de rendimiento.

Paso 1: Abrir el proyecto

```
cd AceriaData
cd src/AceriaData.Console
```

cd AceriaData → entra en la carpeta raíz del proyecto. cd src/AceriaData.Console → entra en la carpeta del proyecto de consola.

Paso 2: Crear el DTO ResumenPorClienteDto

Crear el archivo src/AceriaData.Application/Dtos/ResumenPorClienteDto.cs:

```
namespace AceriaData.Application.Dtos;

public class ResumenPorClienteDto
{
    public string Cliente { get; set; } = string.Empty;
    public int TotalOrdenes { get; set; }
    public int TotalPlanchas { get; set; }
    public decimal PesoTotal { get; set; }
    public DateTime FechaMasReciente { get; set; }
}
```

Línea 1: namespace AceriaData.Application.Dtos; → declara el espacio de nombres.

Línea 3: public class ResumenPorClienteDto → declara el DTO.

Línea 5: public string Cliente { get; set; } = string.Empty; → cliente.

Línea 6: public int TotalOrdenes { get; set; } → total de órdenes.

Línea 7: public int TotalPlanchas { get; set; } → total de planchas.

Línea 8: public decimal PesoTotal { get; set; } → peso total.

Línea 9: public DateTime FechaMasReciente { get; set; } → fecha más reciente.

Error común: si el DTO tiene una propiedad que no se proyecta, EF Core deja la propiedad con su valor por defecto. Se deben proyectar todas las propiedades que se van a usar.

Paso 3: Crear el DTO ResumenPorEstadoDto

Crear el archivo src/AceriaData.Application/Dtos/ResumenPorEstadoDto.cs:

```
namespace AceriaData.Application.Dtos;

public class ResumenPorEstadoDto
{
    public string Estado { get; set; } = string.Empty;
    public int TotalOrdenes { get; set; }
    public decimal PesoTotal { get; set; }
}
```

Línea 1: namespace AceriaData.Application.Dtos; → declara el espacio de nombres.

Línea 3: public class ResumenPorEstadoDto → declara el DTO.

Línea 5: public string Estado { get; set; } = string.Empty; → estado.

Línea 6: public int TotalOrdenes { get; set; } → total de órdenes.

Línea 7: public decimal PesoTotal { get; set; } → peso total.

Error común: si el DTO usa int para el peso total y el valor real es decimal, se produce una pérdida de precisión. Se debe usar decimal para los pesos.

▮ Paso 4: Añadir los métodos de agregación a la interfaz del repositorio

Modificar src/AceriaData.Application/Interfaces/IOrdenRepositorio.cs:

```

using AceriaData.Application.Dtos;
using AceriaData.Domain.Entities;

namespace AceriaData.Application.Interfaces;

public interface IOOrdenRepositorio
{
    OrdenFabricacion? ObtenerPorId(int id);
    List<OrdenFabricacion> ObtenerTodas();
    IQueryable<OrdenFabricacion> ObtenerQueryable();
    OrdenFabricacion? ObtenerPorNumeroOrden(string numeroOrden);
    List<OrdenFabricacion> ObtenerPendientesPorCliente(string cliente);
    List<OrdenFabricacion> ObtenerPorEstadoOrdenadasPorFecha(string estado);
    List<OrdenFabricacion> ObtenerPorRangoDeFechas(DateTime desde, DateTime hasta);
    List<OrdenFabricacion> ObtenerPorClienteOrdenadas(string cliente);
    List<OrdenFabricacion> ObtenerPorClienteYRangoDeFechas(string cliente, DateTime desde, DateTime
hasta);

    List<string> ObtenerClientesUnicos();
    List<OrdenResumenDto> ObtenerResumenes();
    List<OrdenResumenDto> ObtenerResumenesPorEstado(string estado);
    List<OrdenConTotalesDto> ObtenerOrdenesConTotales();
    List<OrdenConPlanchasDto> ObtenerOrdenesConPlanchas();
    List<OrdenConDetalleDto> ObtenerOrdenesConDetalle();
    List<OrdenConTotalesDto> ObtenerOrdenesConTotalesDetallados();
    List<OrdenCompletaDto> ObtenerOrdenesCompletas();

    int ContarOrdenes();
    int ContarOrdenesPorEstado(string estado);
    bool ExisteAlgunaOrden();
    bool TodasLasOrdenesPendientes();
    decimal ObtenerPesoTotalDePlanchas();
    double ObtenerPesoPromedioDePlanchas();
    decimal ObtenerPesoMinimoDePlanchas();
    decimal ObtenerPesoMaximoDePlanchas();
    DateTime ObtenerFechaMasAntigua();
    DateTime ObtenerFechaMasReciente();

    List<ResumenPorClienteDto> ObtenerResumenPorCliente();
    List<ResumenPorEstadoDto> ObtenerResumenPorEstado();
    List<ResumenPorClienteDto> ObtenerClientesConMasDeUnaOrden();

    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
}

```

Línea 1: using AceriaData.Application.Dtos; → importa los DTOs.

Línea 2: using AceriaData.Domain.Entities; → importa las entidades.

Línea 4: namespace AceriaData.Application.Interfaces; → declara el espacio de nombres.

Línea 6: public interface IOOrdenRepositorio → declara la interfaz.

Línea 8: OrdenFabricacion? ObtenerPorId(int id); → método que obtiene por Id.

Línea 9: List ObtenerTodas(); → método que obtiene todas.

Línea 10: IQueryable ObtenerQueryable(); → método que devuelve IQueryable.

Línea 11: OrdenFabricacion? ObtenerPorNumeroOrden(string numeroOrden); → método que obtiene por número.

Línea 12: List ObtenerPendientesPorCliente(string cliente); → método que obtiene las pendientes de un cliente.

Línea 13: List ObtenerPorEstadoOrdenadasPorFecha(string estado); → método que obtiene por estado ordenadas.

Línea 14: `List ObtenerPorRangoDeFechas(DateTime desde, DateTime hasta);` → método que obtiene por rango de fechas.

Línea 15: `List ObtenerPorClienteOrdenadas(string cliente);` → método que obtiene por cliente ordenadas.

Línea 16: `List ObtenerPorClienteYRangoDeFechas(string cliente, DateTime desde, DateTime hasta);` → método que combina cliente y rango.

Línea 18: `List ObtenerClientesUnicos();` → método que obtiene los clientes únicos.

Línea 19: `List ObtenerResumenes();` → método que obtiene los resúmenes.

Línea 20: `List ObtenerResumenesPorEstado(string estado);` → método que obtiene los resúmenes por estado.

Línea 21: `List ObtenerOrdenesConTotales();` → método que obtiene las órdenes con totales.

Línea 22: `List ObtenerOrdenesConPlanchas();` → método que obtiene las órdenes con planchas.

Línea 23: `List ObtenerOrdenesConDetalle();` → método que obtiene las órdenes con detalle.

Línea 24: `List ObtenerOrdenesConTotalesDetallados();` → método que obtiene las órdenes con totales detallados.

Línea 25: `List ObtenerOrdenesCompleatas();` → método que obtiene las órdenes completas.

Línea 27: `int ContarOrdenes();` → método que cuenta las órdenes.

Línea 28: `int ContarOrdenesPorEstado(string estado);` → método que cuenta las órdenes por estado.

Línea 29: `bool ExisteAlgunaOrden();` → método que comprueba si existe alguna orden.

Línea 30: `bool TodasLasOrdenesPendientes();` → método que comprueba si todas las órdenes están pendientes.

Línea 31: `decimal ObtenerPesoTotalDePlanchas();` → método que obtiene el peso total de las planchas.

Línea 32: `double ObtenerPesoPromedioDePlanchas();` → método que obtiene el peso promedio de las planchas.

Línea 33: `decimal ObtenerPesoMinimoDePlanchas();` → método que obtiene el peso mínimo de las planchas.

Línea 34: `decimal ObtenerPesoMaximoDePlanchas();` → método que obtiene el peso máximo de las planchas.

Línea 35: `DateTime ObtenerFechaMasAntigua();` → método que obtiene la fecha más antigua.

Línea 36: `DateTime ObtenerFechaMasReciente();` → método que obtiene la fecha más reciente.

Línea 38: `List ObtenerResumenPorCliente();` → método que obtiene el resumen por cliente.

Línea 39: `List ObtenerResumenPorEstado();` → método que obtiene el resumen por estado.

Línea 40: `List ObtenerClientesConMasDeUnaOrden();` → método que obtiene los clientes con más de una orden.

Línea 42: `void Agregar(OrdenFabricacion orden);` → método que agrega una orden.

Línea 43: `void Eliminar(OrdenFabricacion orden);` → método que elimina una orden.

Error común: si los métodos devuelven `IQueryable`, la capa de aplicación puede componer consultas que el repositorio no controla. Se deben devolver valores simples o DTOs concretos.

■ Paso 5: Implementar los métodos de agregación en el repositorio

Modificar `src/AceriaData.Infrastructure/Repositories/OrdenRepositorio.cs`:

```

public int ContarOrdenes()
{
    return _context.OrdenesFabricacion.Count();
}

public int ContarOrdenesPorEstado(string estado)
{
    return _context.OrdenesFabricacion.Count(o => o.Estado == estado);
}

public bool ExisteAlgunaOrden()
{
    return _context.OrdenesFabricacion.Any();
}

public bool TodasLasOrdenesPendientes()
{
    return _context.OrdenesFabricacion.All(o => o.Estado == "Pendiente");
}

public decimal ObtenerPesoTotalDePlanchas()
{
    return _context.PlanchasAcero.Sum(p => p.Peso);
}

public double ObtenerPesoPromedioDePlanchas()
{
    return _context.PlanchasAcero.Average(p => (double)p.Peso);
}

public decimal ObtenerPesoMinimoDePlanchas()
{
    return _context.PlanchasAcero.Min(p => p.Peso);
}

public decimal ObtenerPesoMaximoDePlanchas()
{
    return _context.PlanchasAcero.Max(p => p.Peso);
}

public DateTime ObtenerFechaMasAntigua()
{
    return _context.OrdenesFabricacion.Min(o => o.FechaCreacion);
}

public DateTime ObtenerFechaMasReciente()
{
    return _context.OrdenesFabricacion.Max(o => o.FechaCreacion);
}

public List<ResumenPorClienteDto> ObtenerResumenPorCliente()
{
    return _context.OrdenesFabricacion
        .GroupBy(o => o.Cliente)
        .Select(g => new ResumenPorClienteDto
        {
            Cliente = g.Key,
            TotalOrdenes = g.Count(),
            TotalPlanchas = g.Sum(o => o.Planchas.Count()),
            PesoTotal = g.Sum(o => o.Planchas.Sum(p => p.Peso)),
            FechaMasReciente = g.Max(o => o.FechaCreacion)
        })
        .OrderBy(r => r.Cliente)
        .ToList();
}

public List<ResumenPorEstadoDto> ObtenerResumenPorEstado()
{
    return _context.OrdenesFabricacion
        .GroupBy(o => o.Estado)
        .Select(g => new ResumenPorEstadoDto
        {

```

```

        Estado = g.Key,
        TotalOrdenes = g.Count(),
        PesoTotal = g.Sum(o => o.Planchas.Sum(p => p.Peso))
    })
    .OrderBy(r => r.Estado)
    .ToList();
}

public List<ResumenPorClienteDto> ObtenerClientesConMasDeUnaOrden()
{
    return _context.OrdenesFabricacion
        .GroupBy(o => o.Cliente)
        .Where(g => g.Count() > 1)
        .Select(g => new ResumenPorClienteDto
        {
            Cliente = g.Key,
            TotalOrdenes = g.Count(),
            TotalPlanchas = g.Sum(o => o.Planchas.Count()),
            PesoTotal = g.Sum(o => o.Planchas.Sum(p => p.Peso)),
            FechaMasReciente = g.Max(o => o.FechaCreacion)
        })
        .OrderBy(r => r.Cliente)
        .ToList();
}

```

Línea 1: public int ContarOrdenes() → declara el método que cuenta las órdenes.

Línea 3: return _context.OrdenesFabricacion.Count(); → cuenta todas las órdenes.

Línea 6: public int ContarOrdenesPorEstado(string estado) → declara el método que cuenta las órdenes por estado.

Línea 8: return _context.OrdenesFabricacion.Count(o => o.Estado == estado); → cuenta las órdenes que cumplen la condición.

Línea 11: public bool ExisteAlgunaOrden() → declara el método que comprueba si existe alguna orden.

Línea 13: return _context.OrdenesFabricacion.Any(); → comprueba si hay al menos una orden.

Línea 16: public bool TodasLasOrdenesPendientes() → declara el método que comprueba si todas las órdenes están pendientes.

Línea 18: return _context.OrdenesFabricacion.All(o => o.Estado == "Pendiente"); → comprueba si todas cumplen la condición.

Línea 21: public decimal ObtenerPesoTotalDePlanchas() → declara el método que obtiene el peso total.

Línea 23: return _context.PlanchasAcero.Sum(p => p.Peso); → suma el peso de todas las planchas.

Línea 26: public double ObtenerPesoPromedioDePlanchas() → declara el método que obtiene el peso promedio.

Línea 28: return _context.PlanchasAcero.Average(p => (double)p.Peso); → calcula el peso promedio.

Línea 31: public decimal ObtenerPesoMinimoDePlanchas() → declara el método que obtiene el peso mínimo.

Línea 33: return _context.PlanchasAcero.Min(p => p.Peso); → obtiene el peso mínimo.

Línea 36: public decimal ObtenerPesoMaximoDePlanchas() → declara el método que obtiene el peso máximo.

Línea 38: return _context.PlanchasAcero.Max(p => p.Peso); → obtiene el peso máximo.

Línea 41: public DateTime ObtenerFechaMasAntigua() → declara el método que obtiene la fecha más antigua.

Línea 43: return _context.OrdenesFabricacion.Min(o => o.FechaCreacion); → obtiene la fecha mínima.

Línea 46: `public DateTime ObtenerFechaMasReciente()` → declara el método que obtiene la fecha más reciente.

Línea 48: `return _context.OrdenesFabricacion.Max(o => o.FechaCreacion);` → obtiene la fecha máxima.

Línea 51: `public List ObtenerResumenPorCliente()` → declara el método que obtiene el resumen por cliente.

Línea 53: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 54: `.GroupBy(o => o.Cliente)` → agrupa por cliente.

Línea 55: `.Select(g => new ResumenPorClienteDto` → proyecta cada grupo al DTO.

Línea 56: `Cliente = g.Key,` → asigna el cliente.

Línea 57: `TotalOrdenes = g.Count(),` → cuenta las órdenes del grupo.

Línea 58: `TotalPlanchas = g.Sum(o => o.Planchas.Count()),` → suma las planchas del grupo.

Línea 59: `PesoTotal = g.Sum(o => o.Planchas.Sum(p => p.Peso)),` → suma el peso de las planchas del grupo.

Línea 60: `FechaMasReciente = g.Max(o => o.FechaCreacion)` → obtiene la fecha más reciente del grupo.

Línea 61: `})` → cierra la proyección.

Línea 62: `.OrderBy(r => r.Cliente)` → ordena por cliente.

Línea 63: `.ToList();` → materializa la consulta.

Línea 66: `public List ObtenerResumenPorEstado()` → declara el método que obtiene el resumen por estado.

Línea 68: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 69: `.GroupBy(o => o.Estado)` → agrupa por estado.

Línea 70: `.Select(g => new ResumenPorEstadoDto` → proyecta cada grupo al DTO.

Línea 71: `Estado = g.Key,` → asigna el estado.

Línea 72: `TotalOrdenes = g.Count(),` → cuenta las órdenes del grupo.

Línea 73: `PesoTotal = g.Sum(o => o.Planchas.Sum(p => p.Peso))` → suma el peso de las planchas del grupo.

Línea 74: `})` → cierra la proyección.

Línea 75: `.OrderBy(r => r.Estado)` → ordena por estado.

Línea 76: `.ToList();` → materializa la consulta.

Línea 79: `public List ObtenerClientesConMasDeUnaOrden()` → declara el método que obtiene los clientes con más de una orden.

Línea 81: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 82: `.GroupBy(o => o.Cliente)` → agrupa por cliente.

Línea 83: `.Where(g => g.Count() > 1)` → filtra los grupos con más de una orden. Se traduce a HAVING.

Línea 84: `.Select(g => new ResumenPorClienteDto` → proyecta cada grupo al DTO.

Línea 85: `Cliente = g.Key,` → asigna el cliente.

Línea 86: `TotalOrdenes = g.Count(),` → cuenta las órdenes del grupo.

Línea 87: `TotalPlanchas = g.Sum(o => o.Planchas.Count()),` → suma las planchas del grupo.

Línea 88: `PesoTotal = g.Sum(o => o.Planchas.Sum(p => p.Peso))`, → suma el peso de las planchas del grupo.

Línea 89: `FechaMasReciente = g.Max(o => o.FechaCreacion)` → obtiene la fecha más reciente del grupo.

Línea 90: `}` → cierra la proyección.

Línea 91: `.OrderBy(r => r.Cliente)` → ordena por cliente.

Línea 92: `.ToList()`; → materializa la consulta.

Error común: si se proyecta una propiedad que no está en el `GroupBy` ni en una función de agregación, EF Core lanza una excepción. Solo se pueden proyectar la clave del grupo o funciones de agregación.

| Paso 6: Crear el caso de uso de agregaciones

Crear el archivo `src/AceriaData.Application/UseCases/AgregacionesUseCase.cs`:

```
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public class AgregacionesUseCase
{
    private readonly IUnidadDeTrabajo _unidad;

    public AgregacionesUseCase(IUnidadDeTrabajo unidad)
    {
        _unidad = unidad;
    }

    public void Ejecutar()
    {
        Console.WriteLine("=== CONSULTAS DE AGREGACIÓN ===");

        DemostrarConteos();
        DemostrarAnyAll();
        DemostrarSumAverageMinMax();
        DemostrarFechas();
        DemostrarResumenPorCliente();
        DemostrarResumenPorEstado();
        DemostrarClientesConMasDeUnaOrden();
        DemostrarSqlAgregacion();
    }

    private void DemostrarConteos()
    {
        Console.WriteLine("\n--- Conteos ---");

        var total = _unidad.Ordenes.ContarOrdenes();
        var pendientes = _unidad.Ordenes.ContarOrdenesPorEstado("Pendiente");

        Console.WriteLine($"Total de órdenes: {total}");
        Console.WriteLine($"Órdenes pendientes: {pendientes}");
    }

    private void DemostrarAnyAll()
    {
        Console.WriteLine("\n--- Any y All ---");

        var existeAlguna = _unidad.Ordenes.ExisteAlgunaOrden();
        var todasPendientes = _unidad.Ordenes.TodasLasOrdenesPendientes();

        Console.WriteLine($"Existe alguna orden: {existeAlguna}");
        Console.WriteLine($"Todas las órdenes pendientes: {todasPendientes}");
    }

    private void DemostrarSumAverageMinMax()
    {
        Console.WriteLine("\n--- Sum, Average, Min y Max ---");
    }
}
```

```

        var pesoTotal = _unidad.Ordenes.ObtenerPesoTotalDePlanchas();
        var pesoPromedio = _unidad.Ordenes.ObtenerPesoPromedioDePlanchas();
        var pesoMinimo = _unidad.Ordenes.ObtenerPesoMinimoDePlanchas();
        var pesoMaximo = _unidad.Ordenes.ObtenerPesoMaximoDePlanchas();

        Console.WriteLine($"Peso total: {pesoTotal} kg");
        Console.WriteLine($"Peso promedio: {pesoPromedio:F2} kg");
        Console.WriteLine($"Peso mínimo: {pesoMinimo} kg");
        Console.WriteLine($"Peso máximo: {pesoMaximo} kg");
    }

    private void DemostrarFechas()
    {
        Console.WriteLine("\n--- Fechas ---");

        var fechaAntigua = _unidad.Ordenes.ObtenerFechaMasAntigua();
        var fechaReciente = _unidad.Ordenes.ObtenerFechaMasReciente();

        Console.WriteLine($"Fecha más antigua: {fechaAntigua:dd/MM/yyyy}");
        Console.WriteLine($"Fecha más reciente: {fechaReciente:dd/MM/yyyy}");
    }

    private void DemostrarResumenPorCliente()
    {
        Console.WriteLine("\n--- Resumen por cliente ---");

        var resmenes = _unidad.Ordenes.ObtenerResumenPorCliente();
        foreach (var resumen in resmenes)
        {
            Console.WriteLine($" {resumen.Cliente} | Órdenes: {resumen.TotalOrdenes} | Planchas: {resumen.TotalPlanchas} | Peso: {resumen.PesoTotal} kg | Última: {resumen.FechaMasReciente:dd/MM/yyyy}");
        }
    }

    private void DemostrarResumenPorEstado()
    {
        Console.WriteLine("\n--- Resumen por estado ---");

        var resmenes = _unidad.Ordenes.ObtenerResumenPorEstado();
        foreach (var resumen in resmenes)
        {
            Console.WriteLine($" {resumen.Estado} | Órdenes: {resumen.TotalOrdenes} | Peso: {resumen.PesoTotal} kg");
        }
    }

    private void DemostrarClientesConMasDeUnaOrden()
    {
        Console.WriteLine("\n--- Clientes con más de una orden ---");

        var resmenes = _unidad.Ordenes.ObtenerClientesConMasDeUnaOrden();
        foreach (var resumen in resmenes)
        {
            Console.WriteLine($" {resumen.Cliente} | Órdenes: {resumen.TotalOrdenes}");
        }
    }

    private void DemostrarSqlAgregacion()
    {
        Console.WriteLine("\n--- SQL generado por una agregación agrupada ---");

        var consulta = _unidad.Ordenes.ObtenerQueryable()
            .GroupBy(o => o.Cliente)
            .Select(g => new
            {
                Cliente = g.Key,
                Total = g.Count(),
                PesoTotal = g.Sum(o => o.Planchas.Sum(p => p.Peso))
            });

        var sql = consulta.ToQueryString();
    }

```

```

        Console.WriteLine(sql);
    }
}

```

- Línea 1: `using AceriaData.Application.Interfaces;` → importa las interfaces de la aplicación.
- Línea 3: `namespace AceriaData.Application.UseCases;` → declara el espacio de nombres.
- Línea 5: `public class AgregacionesUseCase` → declara el caso de uso.
- Línea 7: `private readonly IUnidadDeTrabajo _unidad;` → campo de la unidad de trabajo.
- Línea 9: `public AgregacionesUseCase(IUnidadDeTrabajo unidad)` → constructor.
- Línea 11: `_unidad = unidad;` → asigna el parámetro al campo.
- Línea 14: `public void Ejecutar()` → declara el método principal.
- Línea 16: `Console.WriteLine("=== CONSULTAS DE AGREGACIÓN ===");` → muestra la cabecera.
- Línea 18: `DemostrarConteos();` → llama al método de conteos.
- Línea 19: `DemostrarAnyAll();` → llama al método de Any y All.
- Línea 20: `DemostrarSumAverageMinMax();` → llama al método de Sum, Average, Min y Max.
- Línea 21: `DemostrarFechas();` → llama al método de fechas.
- Línea 22: `DemostrarResumenPorCliente();` → llama al método de resumen por cliente.
- Línea 23: `DemostrarResumenPorEstado();` → llama al método de resumen por estado.
- Línea 24: `DemostrarClientesConMasDeUnaOrden();` → llama al método de clientes con más de una orden.
- Línea 25: `DemostrarSqlAgregacion();` → llama al método de SQL de agregación.
- Línea 28: `private void DemostrarConteos()` → declara el método.
- Línea 30: `Console.WriteLine("\n--- Conteos ---");` → muestra la cabecera.
- Línea 32: `var total = _unidad.Ordenes.ContarOrdenes();` → llama al método del repositorio.
- Línea 33: `var pendientes = _unidad.Ordenes.ContarOrdenesPorEstado("Pendiente");` → llama al método del repositorio.
- Línea 35: `Console.WriteLine($"Total de órdenes: {total}");` → muestra el total.
- Línea 36: `Console.WriteLine($"Órdenes pendientes: {pendientes}");` → muestra las pendientes.
- Línea 39: `private void DemostrarAnyAll()` → declara el método.
- Línea 41: `Console.WriteLine("\n--- Any y All ---");` → muestra la cabecera.
- Línea 43: `var existeAlguna = _unidad.Ordenes.ExisteAlgunaOrden();` → llama al método del repositorio.
- Línea 44: `var todasPendientes = _unidad.Ordenes.TodasLasOrdenesPendientes();` → llama al método del repositorio.
- Línea 46: `Console.WriteLine($"Existe alguna orden: {existeAlguna}");` → muestra el resultado.
- Línea 47: `Console.WriteLine($"Todas las órdenes pendientes: {todasPendientes}");` → muestra el resultado.
- Línea 50: `private void DemostrarSumAverageMinMax()` → declara el método.
- Línea 52: `Console.WriteLine("\n--- Sum, Average, Min y Max ---");` → muestra la cabecera.
- Línea 54: `var pesoTotal = _unidad.Ordenes.ObtenerPesoTotalDePlanchas();` → llama al método del repositorio.

Línea 55: `var pesoPromedio = _unidad Ordenes.ObtenerPesoPromedioDePlanchas();` → llama al método del repositorio.

Línea 56: `var pesoMinimo = _unidad Ordenes.ObtenerPesoMinimoDePlanchas();` → llama al método del repositorio.

Línea 57: `var pesoMaximo = _unidad Ordenes.ObtenerPesoMaximoDePlanchas();` → llama al método del repositorio.

Línea 59: `Console.WriteLine($"Peso total: {pesoTotal} kg");` → muestra el peso total.

Línea 60: `Console.WriteLine($"Peso promedio: {pesoPromedio:F2} kg");` → muestra el peso promedio.

Línea 61: `Console.WriteLine($"Peso mínimo: {pesoMinimo} kg");` → muestra el peso mínimo.

Línea 62: `Console.WriteLine($"Peso máximo: {pesoMaximo} kg");` → muestra el peso máximo.

Línea 65: `private void DemostrarFechas()` → declara el método.

Línea 67: `Console.WriteLine("\n--- Fechas ---");` → muestra la cabecera.

Línea 69: `var fechaAntigua = _unidad Ordenes.ObtenerFechaMasAntigua();` → llama al método del repositorio.

Línea 70: `var fechaReciente = _unidad Ordenes.ObtenerFechaMasReciente();` → llama al método del repositorio.

Línea 72: `Console.WriteLine($"Fecha más antigua: {fechaAntigua:dd/MM/yyyy}");` → muestra la fecha más antigua.

Línea 73: `Console.WriteLine($"Fecha más reciente: {fechaReciente:dd/MM/yyyy}");` → muestra la fecha más reciente.

Línea 76: `private void DemostrarResumenPorCliente()` → declara el método.

Línea 78: `Console.WriteLine("\n--- Resumen por cliente ---");` → muestra la cabecera.

Línea 80: `var resmenes = _unidad Ordenes.ObtenerResumenPorCliente();` → llama al método del repositorio.

Línea 81: `foreach (var resumen in resmenes)` → itera sobre los resúmenes.

Línea 83: `Console.WriteLine($" {resumen.Cliente} | Órdenes: {resumen.TotalOrdenes} | Planchas: {resumen.TotalPlanchas} | Peso: {resumen.PesoTotal} kg | Última: {resumen.FechaMasReciente:dd/MM/yyyy}");` → muestra los datos.

Línea 87: `private void DemostrarResumenPorEstado()` → declara el método.

Línea 89: `Console.WriteLine("\n--- Resumen por estado ---");` → muestra la cabecera.

Línea 91: `var resmenes = _unidad Ordenes.ObtenerResumenPorEstado();` → llama al método del repositorio.

Línea 92: `foreach (var resumen in resmenes)` → itera sobre los resúmenes.

Línea 94: `Console.WriteLine($" {resumen.Estado} | Órdenes: {resumen.TotalOrdenes} | Peso: {resumen.PesoTotal} kg");` → muestra los datos.

Línea 98: `private void DemostrarClientesConMasDeUnaOrden()` → declara el método.

Línea 100: `Console.WriteLine("\n--- Clientes con más de una orden ---");` → muestra la cabecera.

Línea 102: `var resmenes = _unidad Ordenes.ObtenerClientesConMasDeUnaOrden();` → llama al método del repositorio.

Línea 103: `foreach (var resumen in resmenes)` → itera sobre los resúmenes.

Línea 105: `Console.WriteLine($" {resumen.Cliente} | Órdenes: {resumen.TotalOrdenes}");` → muestra los datos.

Línea 109: `private void DemostrarSqlAgregacion() → declara el método.`

Línea 111: `Console.WriteLine("\n--- SQL generado por una agregación agrupada ---"); → muestra la cabecera.`

Línea 113: `var consulta = _unidad Ordenes.ObtenerQueryable() → inicia la consulta.`

Línea 114: `.GroupBy(o => o.Cliente) → agrupa por cliente.`

Línea 115: `.Select(g => new → proyecta a un tipo anónimo.`

Línea 116: `Cliente = g.Key, → incluye el cliente.`

Línea 117: `Total = g.Count(), → cuenta las órdenes.`

Línea 118: `PesoTotal = g.Sum(o => o.Planchas.Sum(p => p.Peso)) → suma el peso de las planchas.`

Línea 119: `}); → cierra la proyección.`

Línea 121: `var sql = consulta.ToString(); → obtiene el SQL.`

Línea 122: `Console.WriteLine(sql); → imprime el SQL.`

Error común: si el método `ObtenerQueryable` no está implementado en el repositorio, el código no compila. Se debe asegurar que la interfaz y la implementación lo incluyen.

| Paso 7: Registrar el caso de uso en el contenedor

Modificar `src/AceriaData.Console/Program.cs`:

```
services.AddScoped<AgregacionesUseCase>();
```

Línea 1: `services.AddScoped(); → registra el caso de uso con ciclo de vida Scoped.`

Error común: si se registra con `Singleton`, se comparte la misma instancia del `DbContext` entre todos los ámbitos y se producen problemas de concurrencia.

| Paso 8: Llamar al caso de uso desde la consola

Modificar el método `Main`:

```
using (var scope = _provider.CreateScope())
{
    var useCase = scope.ServiceProvider.GetRequiredService<AgregacionesUseCase>();
    useCase.Ejecutar();
}
```

Línea 1: `using (var scope = _provider.CreateScope()) → crea un ámbito.`

Línea 3: `var useCase = scope.ServiceProvider.GetRequiredService(); → resuelve el caso de uso.`

Línea 4: `useCase.Ejecutar(); → ejecuta el caso de uso.`

Error común: si se resuelve el caso de uso desde el proveedor raíz en lugar de desde un ámbito, se lanza una excepción `InvalidOperationException`.

| Paso 9: Insertar datos de prueba con varias órdenes y planchas

Asegurarse de que hay datos en la base de datos. Modificar el método `Main` para insertar órdenes y planchas:

```

using (var scope = _provider.CreateScope())
{
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
    context.Database.EnsureDeleted();
    context.Database.EnsureCreated();

    var orden1 = new OrdenFabricacion { NumeroOrden = "OF-2024-0001", Cliente = "Constructora del Norte", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 1, 15) };
    var orden2 = new OrdenFabricacion { NumeroOrden = "OF-2024-0002", Cliente = "Constructora del Sur", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 2, 20) };
    var orden3 = new OrdenFabricacion { NumeroOrden = "OF-2024-0003", Cliente = "Constructora del Norte", Estado = "EnProceso", FechaCreacion = new DateTime(2024, 3, 10) };
    var orden4 = new OrdenFabricacion { NumeroOrden = "OF-2024-0004", Cliente = "Constructora del Norte", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 4, 5) };

    context OrdenesFabricacion.AddRange(orden1, orden2, orden3, orden4);
    context.SaveChanges();

    var plancha1 = new PlanchaAcero { OrdenId = orden1.Id, Espesor = 10.5, Ancho = 1500, Largo = 3000, Peso = 370.5m, Activa = true };
    var plancha2 = new PlanchaAcero { OrdenId = orden1.Id, Espesor = 12.0, Ancho = 1200, Largo = 2500, Peso = 280.8m, Activa = true };
    var plancha3 = new PlanchaAcero { OrdenId = orden2.Id, Espesor = 8.0, Ancho = 1000, Largo = 2000, Peso = 125.6m, Activa = true };
    var plancha4 = new PlanchaAcero { OrdenId = orden3.Id, Espesor = 15.0, Ancho = 1800, Largo = 3500, Peso = 450.0m, Activa = true };

    context.PlanchasAcero.AddRange(plancha1, plancha2, plancha3, plancha4);
    context.SaveChanges();
}

```

Línea 1: `using (var scope = _provider.CreateScope())` → crea un ámbito.

Línea 3: `var context = scope.ServiceProvider.GetRequiredService();` → resuelve el DbContext.

Línea 4: `context.Database.EnsureDeleted();` → elimina la base.

Línea 5: `context.Database.EnsureCreated();` → crea la base.

Línea 7: `var orden1 = new OrdenFabricacion { ... };` → crea la primera orden.

Línea 8: `var orden2 = new OrdenFabricacion { ... };` → crea la segunda orden.

Línea 9: `var orden3 = new OrdenFabricacion { ... };` → crea la tercera orden.

Línea 10: `var orden4 = new OrdenFabricacion { ... };` → crea la cuarta orden.

Línea 12: `context.OrdenesFabricacion.AddRange(orden1, orden2, orden3, orden4);` → registra las órdenes.

Línea 13: `context.SaveChanges();` → inserta las órdenes.

Línea 15: `var plancha1 = new PlanchaAcero { ... };` → crea la primera plancha.

Línea 16: `var plancha2 = new PlanchaAcero { ... };` → crea la segunda plancha.

Línea 17: `var plancha3 = new PlanchaAcero { ... };` → crea la tercera plancha.

Línea 18: `var plancha4 = new PlanchaAcero { ... };` → crea la cuarta plancha.

Línea 20: `context.PlanchasAcero.AddRange(plancha1, plancha2, plancha3, plancha4);` → registra las planchas.

Línea 21: `context.SaveChanges();` → inserta las planchas.

Error común: si se insertan las planchas antes que las órdenes, la clave foránea `OrdenId` no tiene un valor válido y se produce una violación de integridad referencial.

| Paso 10: Ejecutar el proyecto

```
dotnet run
```

dotnet run → compila y ejecuta el proyecto.

Resultado esperado: aparecen las secciones con los resultados de cada agregación. Se observan los conteos, Any y All, las sumas, promedios, mínimos y máximos, las fechas, los resúmenes por cliente y estado, los clientes con más de una orden y el SQL generado por una agregación agrupada.

| Paso 11: Analizar la salida

La salida del programa muestra información como la siguiente:

=== CONSULTAS DE AGREGACIÓN ===

--- Conteos ---

Total de órdenes: 4

Órdenes pendientes: 3

--- Any y All ---

Existe alguna orden: True

Todas las órdenes pendientes: False

--- Sum, Average, Min y Max ---

Peso total: 1226.9 kg

Peso promedio: 306.73 kg

Peso mínimo: 125.6 kg

Peso máximo: 450.0 kg

--- Fechas ---

Fecha más antigua: 15/01/2024

Fecha más reciente: 05/04/2024

--- Resumen por cliente ---

Constructora del Norte | Órdenes: 3 | Planchas: 3 | Peso: 1101.3 kg | Última: 05/04/2024

Constructora del Sur | Órdenes: 1 | Planchas: 1 | Peso: 125.6 kg | Última: 20/02/2024

--- Resumen por estado ---

EnProceso | Órdenes: 1 | Peso: 450.0 kg

Pendiente | Órdenes: 3 | Peso: 776.9 kg

--- Clientes con más de una orden ---

Constructora del Norte | Órdenes: 3

--- SQL generado por una agregación agrupada ---

```
SELECT [o].[Cliente], COUNT(*) AS [Total], COALESCE(SUM((
    SELECT COALESCE(SUM([p].[Peso]), 0.0) FROM [PlanchasAcero] AS [p] WHERE [o].[Id] = [p].[OrdenId]
)), 0.0) AS [PesoTotal]
FROM [OrdenesFabricacion] AS [o]
WHERE [o].[IsDeleted] = CAST(0 AS bit)
GROUP BY [o].[Cliente]
```

La primera sección muestra los conteos. La segunda sección muestra Any y All. La tercera sección muestra las sumas, promedios, mínimos y máximos. La cuarta sección muestra las fechas. La quinta sección muestra el resumen por cliente. La sexta sección muestra el resumen por estado. La séptima sección muestra los clientes con más de una orden. La octava sección muestra el SQL generado por una agregación agrupada.

Observaciones: el SQL generado incluye COUNT(*), SUM, MAX y GROUP BY. El filtro global de Soft Delete se aplica automáticamente. El orden de las cláusulas es el correcto: SELECT, FROM, WHERE, GROUP BY.

Paso 12: Diagnosticar un error común

Modificar el método ObtenerPesoPromedioDePlanchas para llamar a Average sobre una colección vacía:

```
csharp
public double ObtenerPesoPromedioDePlanchas()
{
    return _context.PlanchasAcero
        .Where(p => p.OrdenId == 999)
        .Average(p => (double)p.Peso);
}
```

Resultado esperado: se lanza una excepción InvalidOperationException porque no se puede calcular el promedio de una colección vacía.

Solución: usar DefaultIfEmpty(0) antes de Average o comprobar si hay elementos.

```
public double ObtenerPesoPromedioDePlanchas()
{
    return _context.PlanchasAcero
        .Where(p => p.OrdenId == 999)
        .Select(p => (double)p.Peso)
        .DefaultIfEmpty(0)
        .Average();
}
```

Resultado esperado con la solución: el promedio es cero cuando no hay planchas.

Errores comunes del ejercicio

Error	Causa	Solución
Average sobre colección vacía	No se comprobó si hay elementos	Usar <code>DefaultIfEmpty(0)</code>
Min o Max sobre colección vacía	No se comprobó si hay elementos	Usar <code>DefaultIfEmpty</code> o comprobar
GroupBy sin agregación del grupo o funciones de agregación	Se proyecta una propiedad que no está en el GroupBy	Usar la clave del grupo
Where después del GroupBy para HAVING	Se usa Where en lugar de Having	Usar Where después del GroupBy
Count después de ToList directamente sobre IQueryable	Se materializó la consulta antes de contar	Llamar a Count
Sum sobre tipo anulable	Devuelve <code>null</code> si la colección está vacía	Usar <code>DefaultIfEmpty</code> o COALESCE
Any con <code>Count() > 0</code>	Se usa Count en lugar de Any	Usar Any para mejor rendimiento

Reto: Crear un método en el repositorio que agrupe las órdenes por mes y año, y calcule el total de órdenes, el total de planchas y el peso total por cada grupo. Añadir un DTO, el método a la interfaz, la implementación y una demostración en el caso de uso.

Solución paso a paso:

Paso 1: Crear el DTO ResumenMensualDto:

```
csharp
namespace AceriaData.Application.Dtos;

public class ResumenMensualDto
{
    public int Anio { get; set; }
    public int Mes { get; set; }
    public int TotalOrdenes { get; set; }
    public int TotalPlanchas { get; set; }
    public decimal PesoTotal { get; set; }
}
```

Línea 1: namespace AceriaData.Application.Dtos; → declara el espacio de nombres.

Línea 3: public class ResumenMensualDto → declara el DTO.

Línea 5: public int Anio { get; set; } → año.

Línea 6: public int Mes { get; set; } → mes.

Línea 7: public int TotalOrdenes { get; set; } → total de órdenes.

Línea 8: public int TotalPlanchas { get; set; } → total de planchas.

Línea 9: public decimal PesoTotal { get; set; } → peso total.

Paso 2: Añadir el método a la interfaz IOrdenRepositorio:

```
List<ResumenMensualDto> ObtenerResumenMensual();
```

Línea 1: List ObtenerResumenMensual(); → declara el método.

Paso 3: Implementar el método en OrdenRepositorio:

```
public List<ResumenMensualDto> ObtenerResumenMensual()
{
    return _context.OrdenesFabricacion
        .GroupBy(o => new { o.FechaCreacion.Year, o.FechaCreacion.Month })
        .Select(g => new ResumenMensualDto
        {
            Anio = g.Key.Year,
            Mes = g.Key.Month,
            TotalOrdenes = g.Count(),
            TotalPlanchas = g.Sum(o => o.Planchas.Count()),
            PesoTotal = g.Sum(o => o.Planchas.Sum(p => p.Peso))
        })
        .OrderBy(r => r.Anio)
        .ThenBy(r => r.Mes)
        .ToList();
}
```

Línea 1: public List ObtenerResumenMensual() → declara el método.

Línea 3: return _context.OrdenesFabricacion → inicia la consulta.

Línea 4: .GroupBy(o => new { o.FechaCreacion.Year, o.FechaCreacion.Month }) → agrupa por año y mes.

Línea 5: .Select(g => new ResumenMensualDto → proyecta cada grupo al DTO.

Línea 6: Anio = g.Key.Year, → asigna el año.

Línea 7: Mes = g.Key.Month, → asigna el mes.

Línea 8: TotalOrdenes = g.Count(), → cuenta las órdenes del grupo.

Línea 9: TotalPlanchas = g.Sum(o => o.Planchas.Count()), → suma las planchas del grupo.

Línea 10: PesoTotal = g.Sum(o => o.Planchas.Sum(p => p.Peso)) → suma el peso de las planchas del grupo.

Línea 11: }) → cierra la proyección.

Línea 12: .OrderBy(r => r.Anio) → ordena por año.

Línea 13: .ThenBy(r => r.Mes) → ordena por mes.

Línea 14: .ToList(); → materializa la consulta.

Paso 4: Añadir la demostración en el caso de uso:

```
private void DemostrarResumenMensual()
{
    Console.WriteLine("\n--- Resumen mensual ---");

    var resmenes = _unidad.Ordenes.ObtenerResumenMensual();
    foreach (var resumen in resmenes)
    {
        Console.WriteLine($" {resumen.Anio}-{resumen.Mes:D2} | Órdenes: {resumen.TotalOrdenes} | Planchas: {resumen.TotalPlanchas} | Peso: {resumen.PesoTotal} kg");
    }
}
```

Línea 1: private void DemostrarResumenMensual() → declara el método.

Línea 3: Console.WriteLine("\n--- Resumen mensual ---"); → muestra la cabecera.

Línea 5: var resmenes = _unidad.Ordenes.ObtenerResumenMensual(); → llama al método del repositorio.

Línea 6: foreach (var resumen in resmenes) → itera sobre los resúmenes.

Línea 8: `Console.WriteLine($" {resumen.Anio}-{resumen.Mes:D2} | Órdenes: {resumen.TotalOrdenes} | Planchas: {resumen.TotalPlanchas} | Peso: {resumen.PesoTotal} kg");` → muestra los datos.

Paso 5: Llamar al método desde Ejecutar:

```
DemostrarResumenMensual();
```

Paso 6: Ejecutar dotnet run y verificar que las órdenes se muestran agrupadas por mes y año.

Resultado esperado: las órdenes se muestran agrupadas por año y mes, con el total de órdenes, planchas y peso.

Analogía final

Las consultas de agregación en una acería son como los informes que el jefe de planta pide al archivo central. En lugar de revisar todas las carpetas una por una, el jefe pide un resumen: cuántas órdenes hay, cuánto pesan las planchas, cuál es la fecha más antigua. El archivo central calcula el resumen y lo entrega. Count es como contar las carpetas. Sum es como sumar los pesos. Average es como calcular el promedio. Min y Max son como encontrar el más ligero y el más pesado. GroupBy es como agrupar las carpetas por cliente o por estado. El filtro Having es como pedir solo los grupos que cumplen una condición. Las agregaciones sobre colecciones vacías son como pedir un promedio cuando no hay datos: hay que tener cuidado porque puede dar error. DefaultIfEmpty es como decir "si no hay datos, devuelve cero". Así funcionan las agregaciones en EF Core: se calculan en el servidor, se devuelven resúmenes y se evita transferir todas las filas.

Resultado esperado

Al final del ejercicio, deberías haber:

Creado los DTOs `ResumenPorClienteDto` y `ResumenPorEstadoDto`.

Añadido los métodos de agregación a la interfaz `IOrdenRepositorio`.

Implementado los métodos en `OrdenRepositorio`.

Creado el caso de uso `AgregacionesUseCase`.

Registrado el caso de uso en el contenedor.

Insertado datos de prueba con varias órdenes y planchas.

Ejecutado las demostraciones de conteos, Any, All, Sum, Average, Min, Max, fechas y resúmenes agrupados.

Analizado el SQL generado por una agregación agrupada.

Diagnosticado el error de Average sobre colección vacía.

Creado el DTO `ResumenMensualDto` y su método.

Conexión con el siguiente punto

En este punto se han añadido consultas de agregación al proyecto `AceriaData`, incluyendo Count, Sum, Average, Min, Max, Any, All y GroupBy. Se ha analizado el SQL generado por las agregaciones y se ha comprobado que EF Core traduce las funciones a SQL. En el siguiente punto se estudiarán las agrupaciones con proyección en detalle, incluyendo GroupBy con múltiples claves, filtros Having y proyecciones de grupos.